

Experiment plan: turning the two findings into laws

Version 6, frozen on 17 September 2026. It amends version 5, frozen on 16 September 2026, and stage 0 starts again under it. No run this plan covers starts before it is public (Section 15).

Grey text in brackets is a plain-word explanation. Green tags such as [D6-1] mark what changed since version 5, and grey tags such as [D5-2], [D4-3] and [D3-5] what changed in earlier versions; the four tables below say what each change is and why. Prices are Azure's list prices as read again on 16 September 2026, and they change.

In one paragraph

The paper has three parts, and this plan serves all of them: **the mechanism** of what is happening, **that it is happening**, and **that the industry is not dealing with it**. [D3-1]

Both findings rest on old ideas: late clock readings were shown in 2012, and millisecond rounding is in a 1970 HP manual. What could make them laws is prediction. We write down numbers *before* running, on machines and tools we have never touched, and then check them.

Mode 1 (late notes, the mechanism): we test one specific law, *plateau*, *cliff*, *floor*. The machine's scheduler settings predict the trip length at which negative results suddenly stop, and how sharply they stop.

Mode 2 (rounding and throwing away, the industry): a tool's clock, its rounding and its filter should predict, before it runs, what it will report. Its documentation and issue tracker show whether anyone has dealt with it.

Version 3 added what the pilot on the first Azure machines showed: a calibration of the added delay in every session, a study of why that delay does not lengthen the trip one-for-one, three tick kernels, and the tools block. Version 4 splits the work into 47 small campaigns, each answering one question in a few hours. It sets the number of repeats from a simulation of each campaign's own test, drops what earlier campaigns already answered, and lets campaigns run at the same time on separate machine pairs without mixing machines inside any test. [D4-1]

Version 5 writes down how the campaigns are run. A program judges every run the moment it ends, by rules fixed here, and stops a campaign when the instrument is in doubt. A watch reads every machine pair every five minutes, and every file is fingerprinted from the machine that wrote it to the analysis. No prediction, campaign, number of rounds or decision rule changes. [D5-2]

Version 6 corrects what the first runs of version 5 found, and stage 0 starts again. The calibration must now be *known* within 0.3 ms at every step, where version 5 asked every run to lie within 0.1 ms of it, which no machine could meet. The broker's own packet capture checks the added delay, because ping could not decide 0.05 ms. The Redis client acknowledges messages in batches, as the paper's own settings say, and M0 studies bursty traffic, where the departure it explains appears. The other changes protect runs from faults and keep everything measured. No prediction, and no rule that decides one, changes. [D6-1]

Pocket dictionary

Run	one measurement, a few minutes of traffic	Round	one pass through every setup of a campaign once
Setup	one combination of settings being tested	Session	one machine pair, booted one way, at one core count, on one day
Campaign	runs that answer one question in one sitting, with the machines stopped before and after	Machine pair	a driver machine and a broker machine used together
Time slice (<i>s</i>)	how long the scheduler lets a helper run before switching; 2.8 ms on the first Azure machine	Tick (<i>h</i>)	the kernel's heartbeat; 1 ms if HZ=1000, 4 ms if HZ=250, 10 ms if HZ=100
Trip (<i>D</i>)	the real time from sending to arrival	Plateau, cliff, floor	the three parts of the predicted curve (Section 2)
Anchor slice	the 3 ms slice, included in every slice campaign so that drift between days can be measured and removed	Simulated power	the share of made-up campaigns, generated from the law, in which the decision rule confirms the prediction
Go-first priority	a setting that makes a helper jump the queue (<i>real-time priority</i>)	Receiver-only delay	extra delay added only on the receiving program's connection
Clock offset	making one program's clock read a known amount early or late (<i>libfaketime</i>)	Precision class	low = rounds to milliseconds, medium = microseconds, high = nanoseconds
Retention	the share of results a tool keeps	p50, p99	the median, and the value 99% of results fall below
Pre-register	write the predictions down and freeze them before running	Power	the chance of detecting an effect if it is really there
Calibration	measuring how much a known change moves our reading	Equivalence test	a test that two things are the same within a stated margin (<i>two one-sided tests</i>)
Manipulation check	confirming that the setting we changed really changed	Bridge session	a short repeat that ties new results to earlier ones
Packet capture	a record of network packets with their arrival times	Counterbalanced	arranged so that each kernel comes first, middle and last on different days
Lane	one machine pair with its own queues and logs; two lanes never share a machine	Verdict	what a run's checks decide as it ends: it counts, it is repeated, or it stops its campaign
Stop rule	a reason for a campaign to stop itself and wait for a person	Shakedown	the instrument checks a new machine pair passes before its first campaign
Fingerprint	a SHA-256 code computed from a file; any change to the file changes it	Ledger	the list of a campaign's runs with what happened to each, failures included
Void session	a session broken by a fault: kept and reported, never analysed	Batched acknowledgement	the receiving client confirming many messages in one round trip, instead of one round trip per message
95% interval	a range built so that 95 of 100 such ranges would hold the true value	Bursty traffic	messages sent in clusters, as in the football match, instead of evenly

What changed since version 5, and why

Version 5 was frozen on 16 September 2026 (*freeze 01*), and stage 0 began that day on three machine pairs. Those runs found faults in the machines, in our code and in two of version 5’s own checks, and they were stopped. This version corrects all of them, and stage 0 starts again from the beginning under it. A change prompted by results is exactly what a reader must be able to see, so the table says which result prompted each one. No prediction, and no rule that decides a prediction, changes. The runs of 16 September test nothing; they are kept and reported, and what they showed is the reason for the changes below.

No.	Change	Reason	Prompted by results?
D6-1	P5(b)’s second part asks that the calibration be <i>known</i> within 0.3 ms at every step: its 95% interval there, from the scatter of the runs about the line, or within the steps where the step medians are joined, lies within 0.3 ms of it. Version 5 asked every run to lie within 0.1 ms of the calibration. How far each run and each step median lies from it is still reported.	A limit on every run tests how much runs vary, not the calibration, and more runs make it fail more often. On the second x86 pair, run trips scattered around the calibration with a standard deviation of 0.27 ms (<i>Kafka</i>) and 0.18 ms (<i>Redis</i>). Simulated, 0.1 ms on every run fails every such session, and more than half even at the first pair’s 0.06 ms. A check on the step medians alone would pass by itself whenever the calibration <i>is</i> the joined step medians, which is when a calibration from few runs is least certain. An interval means the same under both forms. Every run is analysed at the trip it actually had (<i>Section 11</i>), so this limit sets how closely runs land where they were aimed, not what they show.	Yes: the second x86 pair’s calibration failed version 5’s check. The first pair’s calibration, 51 of its 56 runs, was also fitted before this version was written.
D6-2	C0 runs in two stages on every pair but the first: 2 rounds, then 2 more, fitted together with the first 2, only when the calibration’s precision (<i>D6-1</i>) is the one part of its gate that failed. The first pair’s staircase S0-1 keeps its 4 rounds.	More rounds only where the data need them, by a rule fixed now. Simulated at the spreads seen: at the first pair’s 0.06 ms no session needs more rounds; at 0.18 ms 16% do, and none then fails; at 0.27 ms 77% do, and 7% then fail. A second stage adds about 1.5 hours to a session.	Yes: follows D6-1 and the second pair’s spread
D6-3	The shakedown’s delay check is made by the broker’s own packet capture: at each step, the broker holds the receiver’s replies for the set delay beyond the host’s, and the host’s no longer than with no delay, within 0.05 ms. Ping still checks the delay end to end, within 0.25 ms plus 5%, the limit every run is held to.	On the Arm pair, one check by ping varied with a standard deviation of 0.03–0.04 ms, and the zero-delay reading drifted by up to 0.1 ms within half a minute. Ping cannot decide 0.05 ms there: the Arm pilot failed the check although the capture shows its delay right to within 0.01 ms. The median of five ping readings would still fail about 8% of sound Arm shakedowns. The capture times the delay where it is added, to a few microseconds, on both processors (<i>between -0.001 and $+0.022$ ms on all three pairs</i>).	Yes: the Arm pair’s shakedown

No.	Change	Reason	Prompted by results?
D6-4	With no delay, the host's and the receiver's ping round trips must agree within 0.25 ms, as the median of five readings. Each pilot keeps both machines' network and timing facts. A pair that passed its shakedown repeats this network part in every later session.	The receiver-only delay assumes the two paths are the same apart from the delay. Captures on both machines put the gap in Azure's network, not in either machine, and it moved between +0.06 and −0.44 ms from one start to the next: a start after a stop can put a machine on another physical host. The limit is the one every run's delay is held to, and it was chosen after the gaps were seen: 0.44 ms on the second x86 pair on 16 September, 0.11 and 0.17 ms on the two x86 pairs on 17 September.	Yes: the second x86 pair's paths, and the probes of 17 September
D6-5	Every run's added delay is measured beyond the same ping taken with no delay just before that run, and the broker's capture of the run's pings is kept beside it.	With no delay, the second x86 pair's paths differed by 0.44 ms, so its zero-delay runs read as −0.44 ms of delay and were repeated. A difference that is there without any delay is not delay. The capture gives the delay to the microsecond at the 16 and 32 ms steps, where ping prints only tenths (<i>D6-13</i>).	Yes: the second pair's first calibration runs
D6-6	Every campaign run uses the paper's own client settings, and each client's log must show that they took, or the run is repeated: Kafka's producer keeps up to 64 requests in flight, as before, and Redis's consumer acknowledges messages in batches of 200, which is new.	The supplement names both settings as learned; the law campaign passed only Kafka's. With one acknowledgement per message, a message waits one round trip for every message read before it. At 16 ms of delay the Redis consumer fell behind to six messages a read and 128 ms trips, which measured its own backlog, not the delay; at 32 ms it would never catch up. On the campaign's steady traffic, up to 8 ms, every read returned one message (<i>6,500 of 6,500</i>), so nothing changes there. Batches of 200 send one acknowledgement every 4 s, which can make at most one message in 200 wait one extra round trip. This setting failed to apply, silently, three times in earlier work; Kafka's producer now writes its setting down too.	Yes: the first pair's 16 ms staircase step

No.	Change	Reason	Prompted by results?
D6-7	M0 replays the football match in bursts, as the pilot did, and changes the Redis consumer's acknowledgement between one per message and batches of 200.	The departure from one-for-one that M0 exists to explain appears only in bursts. In all five pilots, 2 ms of delay lengthened Redis's trip by 2.5–3.2 ms and Kafka's by 1.7–2.0 ms, while on the campaign's steady traffic both x86 calibrations found slopes of 0.99–1.02 up to 8 ms. In the pilot's replay, 46% of sends came under 3 ms after the one before, and 11–13% of Redis reads return several messages, each waiting for the one before it to be acknowledged. M0 as frozen would have run steady traffic and found nothing to explain. Changing the acknowledgement is an intervention on M-H2, the receive loop; M-H1 to M-H4 and their tests are unchanged.	Yes: the calibrations and pilots of 16 September
D6-8	Before every run the driver checks that the receiver's address is only in the receiver's namespace, that the broker answers, and that no package manager is running; a failed check fails the run. Automatic package upgrades are off on every machine, and the receiver's address is removed from the driver's own network settings.	On 16 September an automatic upgrade restarted the first pair's network service during its pilot. The service put the receiver's address back on the driver, the driver lost its broker, and the load read 78.7% instead of 75%. The shakedown failed, as it should.	Yes: a fault of the machines in a pilot
D6-9	Each run's folder carries its campaign's name, and a run never writes into a folder that already exists: the campaign stops instead.	Run keys repeat from one campaign to the next. On 16 September a restarted calibration wrote its first run into the folder of the attempt before it, and the watch showed a 35-minute trip.	Yes: a fault of our code
D6-10	A session broken by a fault, or stopped to fix one, before its calibration's gate is <i>void</i> : kept, copied, and reported with the cause, never analysed. Everything a driver measured outside a campaign (<i>a pilot, a void session, a probe of the instrument, a log</i>) is copied and fingerprinted like a campaign, and a pair's whole folder of runs is copied before the pair is deleted. Every run also keeps the TCP counters of both machines and the broker's own log.	Four sessions were void on 16 September, two on each x86 pair: one broken by the upgrade of D6-8, one by the zero-delay reading of D6-5, and the two running when the fault of D6-9 was found. Kafka runs on both pairs held pauses of 0.15–0.9 s, in 18 of 54 runs and never more than 39 messages, whose cause no file could show; a TCP segment sent again, or the broker pausing, would show in these files.	Yes: faults of the machines and of our code
D6-11	A failed calibration gate ends the session, and the pair starts a new one, with a new C0. The 20-minute deallocation of an idle pair stays; no session is held open while a rule changes.	Version 5 said only that the main runs do not start. Holding a session open across a rule change would judge its data by a rule written after they were seen. A new C0 and B0 cost about two hours.	Yes: a draft of this version would have carried a running session across the change

No.	Change	Reason	Prompted by results?
D6-12	Stage 0 starts again from the beginning on every pair: the shakedown, then S0-1 on the first x86 pair and C0, B0 and P0 on the others. After any change to the code, the first x86 pair runs alone until its shakedown and first calibration pass, and then the others start.	Every run this plan analyses is judged by rules written before it. On 16 September every fault reached all three pairs at once; a staggered start makes a fault in new code cost one pair a few hours.	Yes: the runs of 16 September
D6-13	Delays measured by ping are known to the precision ping prints: 0.001 ms below 1 ms, 0.01 ms from 1 to 10 ms, and 0.1 ms from 10 to 100 ms. The broker's capture (<i>D6-5</i>) gives the finer value at the 16 and 32 ms steps.	All eight runs of the first pair's 16 ms step read the receiver's round trip as exactly 17.0 ms.	Yes: the first pair's staircase

What changed from version 4 to version 5

Version 4 was prepared for freezing but was never frozen, and no run of the campaign has taken place. A change prompted by what a pilot showed is exactly what a reader must be able to see, so every change is listed here and tagged where it appears. Version 5 adds how the campaigns are run, checked, stopped, watched and kept (Section 13). No prediction, campaign, number of rounds or decision rule changes.

No.	Change	Reason	Prompted by pilot results?
D5-1	Every new machine pair passes a shakedown before its first campaign: the pilot's scheduler-settings, receiver-only-delay, load and never-negative checks. Its harness-delay check is replaced by the pair's own C0, and its go-first result is recorded but is not a gate (Section 13).	The first pair's pilot caught a false assumption before any run counted (<i>D3-5</i>). A pair in another region or on another processor gets the same chance to fail before its runs count. Go-first is left to A8, which tests it as a prediction.	Partly: what the first pilot caught
D5-2	Every run is judged as it ends, on its own driver, by fixed limits with three verdicts: it counts; it is repeated, when a condition it was meant to have did not take; or it stops its campaign, when the instrument is in doubt. Two checks are added: at least 99% of the planned messages sent and arrived, and the added delay within 0.25 ms plus 5% of its setting. P5(c)'s stop is applied run by run, against the session's zero-delay "got it" median.	Version 4 listed the integrity checks but not their limits, when they are applied, or what follows a failure. Limits applied by a program cannot be bent after a result is seen, and they hold when nobody is watching.	No
D5-3	A campaign stops itself when its attempts keep failing: the last 3 all failed, or more than 20% of the last 20, once 10 have finished. A run is attempted at most 3 times, and a run whose third attempt fails is abandoned and reported.	A machine that has gone wrong at night should cost minutes, not a night of failed runs.	No

No.	Change	Reason	Prompted by pilot results?
D5-4	A finished campaign's runs are copied from its driver, with a SHA-256 fingerprint for every file checked after the copy, before the next campaign starts on that pair. The fingerprints, and a record of the machine, the code version and the time, are kept with the data. A pair is deleted only after all its campaigns are copied.	Results get lost silently in copies. A fingerprint makes a truncated or changed file visible, and copying after every campaign means a lost machine costs at most the campaign in progress.	No
D5-5	A watch outside the testbed reads every pair every 5 minutes, raises alerts, warnings and notices, and deallocates a pair idle for 20 minutes.	Campaigns run unattended, often at night. A stopped campaign or a broken machine is seen within minutes, and idle machines stop billing.	No
D5-6	Every run records its pair, its driver and the version of the code (<i>the commit</i>) that ran it. All pairs run the same version, and a campaign's queue runs on one pair only.	D4-10 records the pair. Recording the code version too ties every result to the exact code behind it, and one version on every pair keeps the pairs comparable.	No
D5-7	All three machine pairs run in Sweden Central, so the second x86 pair does not go to Italy North. The subscription's limits there were raised on 16 September 2026 to 40 CPUs in total, 20 in the x86 family and 10 in the Arm family.	One region is cheaper (<i>EUR 0.4165 an hour a pair against 0.4577</i>), puts both x86 pairs on the same hardware generation, and lets all three pairs run at once, about 5 days. The prices were read again on 16 September and had not moved.	No

What changed from version 3 to version 4

Version 3 was approved but not frozen.

No.	Change	Reason	Prompted by pilot results?
D4-1	The work is split into 47 small campaigns, each answering one question in one sitting (<i>3.5 to 8.4 hours at the floor of 4 rounds</i>), with the machines stopped between them (Section 12). Version 3's two stages are replaced.	A design decision, 15 September: results arrive in pieces that can each be checked, and no machine waits idle between blocks.	No
D4-2	Rounds are set for each campaign by simulating its own prediction and decision rule at the run-to-run spread its machine pair measured, with a floor of 4 and a ceiling of 40. Tool campaigns run 4 rounds. This replaces version 3's fixed floors of 15 and 10 (Section 11).	The fixed floors came from old setups whose spread was about as large as the rate itself. The pilot's repeat runs varied far less, and simulating the actual decision rule measures power directly.	Yes: the pilot's run-to-run spread
D4-3	A1 runs as six campaigns, three per backend, each including the 3 ms slice as a shared anchor. Each campaign's offset is removed through the anchor before slices are compared.	The slices no longer share one session; the anchor keeps the comparison fair across days.	No

No.	Change	Reason	Prompted by pilot results?
D4-4	A3 tests one slice (<i>3 ms</i>) instead of two, in two campaigns, one per backend. P3 is judged at that slice.	Earlier campaigns (<i>E-A3</i> , <i>E-A6</i>) already showed load raising the negative rate. What is new is whether load moves the cliff, and one slice tests that.	No
D4-5	A7 tests one slice (<i>3 ms</i>), both backends, in one campaign. P4 is judged at that slice.	Go-first priority was shown on the earlier machines (<i>E-A5</i>) and again, 27-fold, in the pilot on this machine. What is new is its effect at the cliff.	Partly: the pilot reproduced go-first
D4-6	A5 runs as two campaigns, one per backend. Each core count is its own session with its own calibration.	Switching CPUs off changes the machine, so the delay's effect is measured again.	No
D4-7	A2 runs as a build campaign, twelve sessions (<i>kernel × day × backend</i>) and two bridge sessions on the stock kernel.	Sessions of about 3.5 hours, with each kernel on two days in a balanced order.	No
D4-8	A4 opens with its own instrument campaign on the Arm pair (<i>C0</i> , <i>B0</i> and <i>P0</i>), then four campaigns: two per backend, at slices of 1.5 and 3 ms, and of 3 and 4.5 ms.	The repeats rule needs each pair's own spread (<i>D4-2</i> , <i>D4-10</i>).	No
D4-9	A8 is kept, narrowed to Kafka, whose official client is written in Java, in two campaigns: one on an x86 pair and one on the Arm pair. P8 says how the cliff is judged at three trips.	The earlier client comparison (<i>M1</i>) tested a second Python client on a different delay, so it does not answer P8.	No
D4-10	Several machine pairs may run at the same time. Every prediction is tested on runs from one pair, every pair opens with its own B0 and P0, and every run records its pair.	A shorter calendar, without mixing machines inside any test.	No
D4-11	Straight reruns of earlier campaigns are dropped. A new section lists what earlier campaigns already answered and what is still run (Section 6).	Nothing is repeated that a new prediction does not need.	Partly: the pilot reproduced go-first
D4-12	Stage 0 opens with a staircase campaign (<i>S0-1</i>): added delays up to 16 ms, both backends, 4 rounds, before any other campaign.	A two-round session calibration has little power to test whether the line is straight; this campaign has more, and it shows what later calibrations should look like.	Yes: the pilot's delay result
D4-13	The plan is frozen by committing it in full to the public repository before the first Stage 0 run; a later change is a new, numbered freeze (Section 15).	Version 3 listed three ways and left the choice open.	No
D4-14	P9 is restated for what A4 measures: three slices at one load, with the anchor rule of A1.	Version 3's P9 also named P2 and P3, which A4 never tested. A prediction must match its experiment.	No

What changed from version 2 to version 3

No.	Change	Reason	Prompted by pilot results?
D3-1	The paper’s three parts frame the plan, and the tools block (<i>T1–T4</i>) is in this paper.	A design decision, 15 September.	No
D3-2	Azure only. A4 compares Azure’s Arm machine with its x86 machine; the Oracle options are dropped.	A design decision, 14 September.	No
D3-3	Default slices are predicted from the per-CPU constant read off each machine, not from the kernel’s version (<i>A5, P7</i>).	The Azure 6.8 kernel carries the constant Linux 6.15 adopted: 2.8 ms at 8 CPUs, where the 6.8 rule gives 3 ms. A reading of the machine’s settings, not an experimental result.	No
D3-4	New section: what the pilot showed.	A record of the pilot.	–
D3-5	Version 2’s built-in delay check is replaced by a calibration in every session (<i>C0</i>). The “got it” check becomes an equivalence test. The law is analysed at each run’s measured trip. P5 is rewritten.	The pilot showed the trip does not grow one-for-one with the added delay, and in opposite directions for the two clients.	Yes
D3-6	New study M0: why the trip does not grow one-for-one, with four explanations and the test that decides each.	The same pilot result. A law’s measuring setup has to be understood, not only corrected.	Yes
D3-7	A2 uses three kernels (<i>ticks of 1, 4 and 10 ms</i>) built by us from one source, with a bridge to the stock kernel, checks at every boot and counterbalanced sessions. P2b, P2c and P2d are added.	Three levels test a proportion, not only a direction.	No
D3-8	New T5: an audit of each tool’s documentation and issue tracker, and prediction T-P8.	It turns “the industry is not dealing with it” into a measurement.	No
D3-9	Size and schedule corrected: each backend counts as its own runs, and a run takes 3.5 minutes as the pilot measured, not 1.5.	Version 2 underestimated the machine time about four-fold.	Pilot timing only
D3-10	How the plan is frozen is specified.	Version 2 said “committed with a date and hash” without saying where. Settled by D4-13.	No

1 The three parts, and the questions they ask [D3-1]

1. **The mechanism (Mode 1).** Is there a law that predicts, from the machine’s scheduler settings alone, for which trip lengths a latency measurement goes negative? Section 2 states the one we test.
2. **That it is happening.** Does the effect appear on machines, processors and kernels it was never seen on? Every run of this campaign measures it on new hardware, and A4 moves to a different processor family.
3. **That the industry is not dealing with it (Mode 2).** Does a tool’s source code predict what it reports? We check whether it rounds, whether it averages over negatives, whether it drops or replaces odd values, and whether it measures the whole trip at all. We test this on every free, widely used tool we can run, not just the one we audited. And we check whether its documentation says so, and what happened when users reported it.

2 The law we are testing for Mode 1: plateau, cliff, floor

The idea in plain words. Most of the time the helper that writes down “got it” gets a core almost at once. But sometimes another task is holding the core. The scheduler lets that task finish its protected time slice, plus up to one tick, before the helper runs. The kernel’s own developers state this bound: the wait stays within “slice plus tick”. So a kept-waiting helper waits roughly between one slice and one slice plus a tick. That gives three parts:

- **Plateau.** If the trip is shorter than the slice, almost every long wait outlasts it, and the negative rate stays roughly flat.
- **Cliff.** Between one slice and one slice plus a tick, fewer and fewer waits outlast the trip, so the rate falls roughly in a straight line.
- **Floor.** For trips longer than one slice plus a tick, long waits almost never outlast the trip. Only a small floor remains, from ordinary short delays.

The same thing as a formula. Write s for the time slice, h for the tick, p for how often the helper is kept waiting, and c for the floor. Then

$$\text{negative rate}(D) \approx c + p \times \begin{cases} 1 & D \leq s \quad (\text{plateau}) \\ \frac{s + h - D}{h} & s < D < s + h \quad (\text{cliff}) \\ 0 & D \geq s + h \quad (\text{floor}) \end{cases}$$

What the settings predict, and what we still have to measure.

- **Predicted from settings alone:** *where* the cliff starts, at s , and *how wide* it is, h . Both can be read from the machine, or set by hand, before any run.
- **Measured, not predicted:** *how high* the plateau is, p , which rises with load, and the floor c . Very short trips, under about 0.5 ms, also pick up short waits the formula ignores. So the tests below focus on where the cliff sits and how wide it is.

Does our old data fit? On the old 8-core machines ($s = 3$ ms, $h = 1$ ms), 13% of traced waits were over 2 ms but only 2.8% over 4 ms. That is the cliff, between 3 and 4 ms. It fits, but it is one machine, and a law has to survive new ones. What the first Azure pilot saw is in Section 5.

How it could fail. It fails if the cliff does not move when we change the slice (P1). It fails if its width does not follow the tick (P2 to P2c). And it fails if load moves the cliff rather than just raising the plateau (P3).

3 Machines and money [D3-2] [D4-10]

Pair	Machines and region	Price while running
First x86 pair	Driver Standard_D8as_v6 (8 CPUs, AMD EPYC Genoa) and broker D2as_v6 (2 CPUs), in Sweden Central. The pilot ran here.	about EUR 0.42 an hour
Second x86 pair	The same sizes, in Sweden Central beside the first pair, in its own resource group and network. [D5-7]	about EUR 0.42 an hour
Arm pair	Driver D8ps_v6 (8 CPUs, Azure Cobalt 100) and broker D2ps_v6, in Sweden Central, the only region that offers these sizes to this account. For A4 and A8.	about EUR 0.32 an hour

All pairs live in one pay-as-you-go Azure subscription. Each region allows a set number of CPUs, and a stopped machine still counts against that limit. On 16 September 2026, on request, Sweden

Central’s limits were raised to 40 CPUs in total, 20 in the x86 family and 10 in the Arm family, so all three pairs fit in that one region. [D5-7] A stopped machine costs only its disk and its fixed address, about EUR 26 a month per pair. A pair that will not be used for more than about two weeks is deleted and later recreated, which takes about 15 minutes.

Every tool below is open source and free. Each tool’s licence is recorded when it is downloaded.

4 The tools

Precision classes come from reading each tool’s code. Tools marked “read first” are classified before any run, and that classification is itself a prediction (Section 10). Each tool’s version is pinned by release or commit before any run.

Tool	Times	Class	Odd values (zero or negative), from source	Runs on
OpenMessaging Benchmark	send → receive; send → “got it”	Low; High	end-to-end: drops ≤ 0 , uncounted. Publish: keeps all	Kafka, Valkey, RabbitMQ, NATS
Kafka EndToEndLatency	send → receive	Low (chops)	keeps all; average exact	Kafka
Kafka ProducerPerformance	send → “got it”	Low	keeps all	Kafka
Apache Pulsar perf	send → receive	Low	drops negatives, keeps zeros, uncounted	Pulsar
emqtt-bench	send → receive	Low	keeps all	EMQX or Mosquitto
wrk2	request → response	Medium	negatives never recorded; may crash	nginx
RabbitMQ PerfTest	send → receive	High	keeps all, negatives included	RabbitMQ
NATS CLI (latency)	send → receive	High	keeps all	NATS server
<i>Read first, then run (widely used, class not yet known)</i>				
nats bench	to confirm	?	to confirm	NATS server
valkey-benchmark (<i>free Redis fork</i>)	request → reply	?	to confirm	Valkey
mentier_benchmark	request → reply	?	to confirm	Valkey
librdkafka rdkafka_performance	send → “got it”?	?	to confirm	Kafka
YCSB, k6, Vegeta, hey	request → reply	?	to confirm	nginx or Valkey
<i>Kept as source readings only (they do not time a message trip)</i>				
fio, btt, Rezolus	disk and scheduler timings	High	fio replaces negatives with 0; btt with 1 ns; Rezolus counts its discards	–

Our own program is the reference instrument everywhere. It reads nanoseconds, keeps everything, and records all three times.

5 What the pilot showed [D3-4]

The first pilot ran on 14 and 15 September on the first x86 pair, at 75% load. A pilot tests the instruments, never a prediction.

Check	Rule written before the pilot	Result
Scheduler settings	a slice set by hand is still set 60 s later, and the default comes back	Passed: 1.5 ms held; the 2.8 ms default restored
Receiver-only delay, by ping	the set delay reaches the receiver’s address only	Passed: 0.5 ms measured as 0.515 ms, 2.0 ms as 2.03 ms
Load	within 3 points of 75%	Passed: 75.3% in all three cells
Never negative	arrival never before sending	Passed: none in 18 runs, about 20,000 messages
Delay seen by the harness	the trip grows by the added 2.03 ms within 0.05 ms, and “got it” moves under 0.05 ms	Failed. Kafka’s trip grew 1.80 ms and “got it” moved -0.14 ms. Redis’s trip grew 2.51 ms and “got it” moved -0.02 ms.
Go-first priority	cuts the negative rate at least 5-fold, intervals apart, at the same load	Passed: 13.5% to 0.50% (<i>27-fold</i>), at 75.3% and 75.4% load

What the failed check means. The delay does reach only the receiver: ping says so, and “got it” barely moves. But the trip does not grow one-for-one with it. Repeat runs agree within about 0.1 ms (*Kafka’s trips were 5.29–5.40 ms with the delay and 3.48–3.92 ms without; Redis’s 4.56–4.72 and 2.06–2.33*), so this is a property of the two clients, in opposite directions, and not noise. Version 2’s rule was an assumption, and it is false. Trips placed by “delay = target minus baseline” would miss by up to half a millisecond, as much as the detail of the cliff we want to see. Hence C0 and M0 in Section 7. [D3-5] [D3-6] The pilot replayed the football match at ten times its speed, in bursts. On the campaigns’ steady 50 messages a second, the calibrations of 16 September found both clients tracking the delay within 2% up to 8 ms, so the departure belongs to bursty traffic, and M0 studies it there. [D6-7]

The machine’s own settings. Kernel 6.8.0-1064-azure, tick 1 ms ($HZ=1000$). The default slice was 2.8 ms on the 8-CPU driver and 1.4 ms on the 2-CPU broker. That is the per-CPU constant of Linux 6.15, not the 3 and 1.5 ms the 6.8 rule gives. [D3-3]

Consistent with the law, but not a test. Without the delay, Kafka’s trip (*about 3.5 ms*) sat inside the cliff the law predicts for this machine (*2.8 to 3.8 ms*), and Redis’s (*about 2.2 ms*) sat on the plateau. With 2 ms added, both trips passed the start of the floor, and negative results fell from 7–12% of messages to 1–1.5% for both clients. That is the direction the law predicts. But pilot data never tests a prediction, and two points are not a curve.

6 What earlier campaigns already answered [D4-11]

Our earlier campaigns ran on Oracle Cloud machines of the same AMD generation. Where one of them already answers a question, it is not run again, and only what a new prediction needs is run.

Earlier campaign	What it showed	What this plan still runs
E-A5, and the Azure pilot	Go-first priority on the timestamping helper cuts negative results many times over at equal load; 27-fold in the pilot.	Only its effect at the cliff: A7, at one slice.
E-A3 and E-A6	Negative results rise with load; E-A6 also tested whether it matters where on the machine the load sits, at equal utilisation.	Only whether load moves the cliff: A3, at one slice.
E-A9	Helper waits traced from the kernel’s scheduler events predicted the negative rate, with ratios of prediction to measurement between 0.78 and 1.32 in ordinary runs.	Rides along in half of the A1 and A3 runs at no extra cost (<i>A6, P6</i>), on new hardware.

Earlier campaign	What it showed	What this plan still runs
E-A10	Lengthening the true trip by padding messages lowered the negative rate (<i>4.1-fold for a 77-fold longer trip</i>) without touching the scheduler.	The slice campaigns (<i>A1</i>) place the cliff; the padding sweep is not rerun.
E-X to E-X4	Four audits of the OpenMessaging Benchmark: its end-to-end result drops values at or below zero without counting them.	No new runs of that tool; its source reading stays in the tools table.
M1	A second Kafka client for Python (<i>confluent-kafka</i>), compared on a different delay: the producer’s scheduling lag at real-time replay.	It does not answer P8, so A8 compares Python with Java at the cliff.
Earlier mechanism campaigns, rerun unchanged	–	Not run. The pilot already reproduced go-first on this machine, and every campaign here measures the effect on new hardware.

7 Experiments for Mode 1 (the mechanism)

The fix: slow only the receiver. Our sender and receiver sit on the same machine, so the broker’s replies to both travel the same link. A delay added to that whole link slows the “got it” reply as much as the message. Both times then move together and the test sees nothing. That is exactly why our old delay sweep failed. So:

1. Give the receiving program its own network address on the client machine (*a second IP address, or its own network namespace*), and make it connect from that address only.
2. On the broker machine, add the delay only to traffic going to that address. Replies to the sender’s address are untouched.
3. **Checked, not assumed:** the delay’s effect on the trip is measured in every session (C0 below), and the “got it” delay must stay equivalent (P5). The added delay itself is measured on its own at every step, by ping from both sides beyond the same ping taken with no delay just before, and by the broker’s own capture of those pings, and the measured value is used, not the setting. [D3-5] [D6-5]

Always the same: background load is measured, not just set. Before every run we read back the machine’s actual time slice (*the kernel’s base-slice setting*) and tick (*CONFIG_HZ*), and record its core count, CPU model, kernel version and machine pair. [D4-10]

C0. Calibrating the added delay, in every session. [D3-5] *Why:* the pilot showed that adding 2.03 ms on the receiver’s path lengthened the trip by 1.80 ms for Kafka and by 2.51 ms for Redis, repeatably. The trip is what the law is about, so the delay needed to reach a planned trip has to be measured, not assumed.

- **Steps:** added delays of 0, 1, 2, 4 and 8 ms, plus 16 ms in sessions whose trips reach that far and 32 ms in HZ=100 sessions. The zero step runs twice per round, to catch drift. Each step’s delay is measured by ping from both sides, and the measured value is used. Ping prints tenths of a millisecond above 10 ms, so the broker’s capture is kept beside it. [D6-13] At the 32 ms step, reads come back less often than messages arrive, so trips there do not grow one-for-one for either client. [D6-6]
- **Order and repeats:** each backend and each load the session runs, 2 rounds before the session’s main runs, shuffled as in Section 11, and 2 more when the calibration is too loosely known and nothing else is wrong. [D6-2]

- **First, a staircase campaign (S0-1):** [D4-12] before any other campaign, the steps up to 16 ms run for both backends at 75% load for 4 rounds, to test with more power whether the line is straight.
- **What it gives:** for each backend, a line of run median trip against measured delay. Each main run’s delay is read off that line for its target trip.
- **Checked all session long:** every main run also records its delay and its trip. At analysis the line is refitted from all of the session’s runs, and its slope is compared across slices and loads.
- **Gate:** a session’s main runs start only if P5(b) holds. Otherwise the session ends, and the pair starts a new one with a new C0. [D6-11]

M0. Why the trip does not grow one-for-one (campaign S0-2). [D3-6] *Why:* a law about timestamps needs its measuring setup understood, not just corrected. The departure appears when messages come in bursts (*the pilot’s replay*), not on the campaigns’ steady traffic. [D6-7] The four candidate explanations, and the test that decides each, are M-H1 to M-H4 in Section 10.

- **Setups:** the football match replayed in bursts, as in the pilot; both backends, added delays of 0, 2 and 8 ms, at 75% load on the first x86 pair, 10 rounds; Redis both with one acknowledgement per message and in batches of 200, an intervention on M-H2. [D6-7]
- **Recording, in half the runs:** a packet capture on the receiver’s own address and on the broker; the times at which our receiving program issues each fetch or read and at which it returns; and the receiving thread’s wake-up delays (*with the tracing tool the harness already uses for the “got it” helper*). The other half runs without recording, so the recording’s own effect is measured, as in A6.
- **Afterwards:** the request-timing log stays on in later runs only if the recorded and unrecorded halves differ by less than 0.02 ms in median trip and median “got it” delay.

Trip lengths are placed around the predicted cliff. For each slice s and tick h we use 8 trips: two on the plateau ($0.5s$ and $0.9s$), four across the cliff ($s + 0.2h$, $s + 0.4h$, $s + 0.6h$, $s + 0.8h$), and two beyond it ($s + 1.5h$ and $2(s + h)$). Each trip is reached with the delay the session’s calibration says it needs, and the analysis uses the trip each run actually had. [D3-5]

A1. Dose–response on the slice (the main test). On the first x86 pair, set the time slice by hand to six values: 0.75, 1.5, 2.25, 3, 4.5 and 6 ms, at 75% load. *Question:* does the cliff move with the slice, one for one?

- **Six campaigns, three per backend:** [D4-3] slices of 0.75, 3 and 4.5 ms; of 1.5, 3 and 6 ms; and of 2.25 and 3 ms; 8 trips each.
- **The anchor:** the 3 ms slice is in every campaign. Compared across campaigns, its halfway point and plateau height measure each campaign’s offset, which the analysis removes before slices are compared (*Section 11*).
- All six run on one machine pair, in an order drawn with a recorded seed.

A2. The tick, with three kernels we build ourselves. [D3-7] The tick is fixed when the kernel is compiled, so it cannot be changed on a running machine. *Question:* does the cliff’s width follow the tick, in proportion?

- **One source, three builds.** The driver’s own kernel (*6.8.0-1064-azure*), from Ubuntu’s source package and with the running kernel’s own configuration, is built three times on the same machine with the same compiler, changing only the tick: HZ=1000, 250 and 100 (*ticks of 1, 4 and 10 ms*). Our HZ=1000 build is the control, so the three kernels differ in nothing else. Three levels test a proportion; two could only show “wider”.

- **A bridge to everything else.** Short sessions on Azure’s stock kernel run the same setups as our HZ=1000 build, so the tick results connect to A1, A3 and A5, which run on the stock kernel. They should agree within 0.1 ms at the cliff’s halfway point.
- **Only the driver changes.** The broker keeps its kernel throughout, because the helper that writes “got it” runs on the driver.
- **Checked at every boot, before any run** (*manipulation checks*): the tick setting read back from the running kernel; the tick measured by counting the kernel’s tick timer on a busy CPU for 10 s, within 5% of the setting; the scheduler’s high-resolution slice timer (*HRTICK*) switched off in every kernel, because with it on the tick would not set the width and the test would be void; the slice read back as set; and the same tickless settings (*NO_HZ*) in all three builds.
- **Booting safely.** A new kernel is booted once while the stock kernel stays the default, so a failed boot falls back to it on the next restart. Azure’s serial console is the last resort.
- **Campaigns.** [D4-7] A build campaign compiles the three kernels and boots each once with its checks. Then twelve sessions, one kernel and one backend each, over four days: each kernel runs with each backend on two different days, and across the days each kernel runs first, middle and last (*a Latin square*). Two bridge sessions, one per backend, run on the stock kernel. Each session starts with its own C0 and a short B0 at 75% load, because the tick may change both the delay’s effect and the baseline trip. All fifteen campaigns run on one machine pair.
- **Setups:** 2 slices (*1.5 and 3 ms*) $\times$ 8 trips placed with that kernel’s tick, one backend per session. Each session runs half of the block’s rounds, so each kernel and backend gets the full number over its two days.

A3. Load. [D4-4] Run at 50%, 75% and 88% load, at the 3 ms slice and 6 trips, in two campaigns, one per backend. Each campaign calibrates at all three loads. *Question:* does load raise the plateau without moving the cliff?

A4. Another kind of machine. [D3-2] [D4-8] On Azure’s Arm machine (*Azure Cobalt 100*): first an instrument campaign on the Arm pair (*C0, B0 and P0*), then four campaigns, two per backend, at slices of 1.5 and 3 ms and of 3 and 4.5 ms, 8 trips each. The 3 ms slice is the anchor, as in A1. *Question:* does the cliff follow the slice on a processor the law never saw?

A5. Default slice from core count (secondary). [D3-3] [D4-6] On an x86 driver with 2, 4 and 8 CPUs switched on, left at their default slices, check that the kernel’s default follows its rule and that the cliff follows it. The rule is computed from the per-CPU constant each machine reports, not from its version: the first Azure driver’s 6.8 kernel carries the constant Linux 6.15 adopted, which gives 1.4, 2.1 and 2.8 ms where the 6.8 rule gives 1.5, 2.25 and 3 ms. Two campaigns, one per backend. Each core count is its own session, with its own C0, at 8 trips.

A6. Predict from a recording, with nothing fitted. In a random half of A1 and A3 runs, record the helper’s waits in 0.1 ms steps, and predict the negative rate from that recording. Tracing slows things slightly, so traced and untraced runs are compared too.

A7. Go-first priority. [D4-5] At the 3 ms slice and three trips (*0.9s, s+0.5h and s+1.5h*), turn go-first priority on for the timestamping helpers only, at the same load. Both backends, in one campaign.

A8. Client language. [D4-9] Kafka only: our Python client against Kafka’s official Java client, with the “got it” note taken on the background helper or on the sending thread, at the 3 ms slice and the same three trips as A7, ordinary and go-first. One campaign on an x86 pair and one on the Arm pair. Each client has its own calibration, because the delay’s effect on the trip belongs to the client.

8 Experiments for Mode 2 (the industry)

Each runnable tool is one campaign on one machine pair, with our own program recording the same messages. [D4-1] For each tool, at least two experiments, and three where it applies:

T1. Delay staircase (*receiver-only, as in the fix above*). Add 0, 0.1, 0.2, 0.3, 0.5, 0.7, 0.9, 1.1, 1.5 and 2.0 ms, in random order. Record the tool’s average, p50 and p99, and our reference at each step. *Questions:* does the tool see a 0.1 ms step? Does it round, chop or keep? Does its average track the truth?

T2. Forced negatives. Make the receiver’s clock read 0.5 ms and 2 ms early (*libfaketime for Java, C and Python tools, checked on each tool first; a second machine with a deliberately offset clock for Go tools, restored afterwards*). *Questions:* does it average over the negatives, drop them, replace them with zero, crash, or count them? Does its message count still match what was sent?

T3. Load and go-first priority. Run each tool on an idle machine and at 88% load, with and without go-first priority on the tool’s own process. *Question:* how much of the tool’s own number is waiting in line?

T4. What it measures. Check which trips each tool can report at all: send to receive, send to “got it”, or only request to reply. Confirm from the documentation, the source and a real output file.

T5. Documentation and issue audit (*desk work, no machines*). [D3-8] For each tool in Section 4:

1. Read its documentation for three things: which clock and resolution it times with; what it does with zero or negative values; and any warning about scheduling or clock effects.
2. Search its issue tracker and mailing list with fixed terms: “negative latency”, “negative”, “clock skew”, “resolution”, “rounding”, “millisecond”, “nanosecond” and “latency 0”.
3. Record every relevant report with its date and outcome: fixed, closed without a fix, or still open.

The terms and the reading rules are frozen with this plan. Results are counts per tool, with every report linked. *Question:* where a tool mishandles values, does anyone say so, and did anything change when users noticed?

9 What is already known

Mode 1: no law found.

- **The kernel’s rule is only a bound.** The developers state a bound, slice plus tick, not a distribution. Their 2026 conference talk measured wakeup delays and explicitly presented no model.
- **Classic waiting-time laws cover jobs, not timestamps.** Kleinrock (1967) and Coffman, Muntz and Trotter (1970) give waiting-time laws under time slices for jobs in a queue; nobody has applied them to timestamps.
- **Nearby laws cover other things.** Tsafir (2005) models noise from kernel ticks in parallel programs. Sharma (2026) gives a condition for clock differences, not for scheduling.
- **Timestamping and tracing studies are empirical.** Villain (2012), Giraldeau and Dagenais (2015) and Bielmeier (2025) measure delays or predict them from recordings, with no formula from settings.

Mode 2: comparisons exist outside brokers. OctoPerf (2022, practitioner blog) found four HTTP tools reporting averages from 20 to 150 ms on one site. Lancet (2019) found three load testers reporting 99th percentiles of 27, 40 and 63 μ s. Every broker comparison found uses one tool per study, and none checks rounding, negatives or discards.

Practitioner sources (ordinary web search).

- **Why tools disagree.** Practitioner write-ups agree that tools report different latencies mainly because they start and stop the clock in different places. OctoPerf’s comparison of four HTTP tools is the clearest case. The “coordinated omission” write-ups make the same point about *when* the clock starts: Gil Tene’s wrk2, Tyler Treat’s 2016 message-queue benchmarks, ScyllaDB (2021) and the YCSB fix. Kroxylicious (2026) notes that Kafka’s producer perf test cannot show end-to-end latency at all.
- **Millisecond clocks.** It is widely known that a millisecond clock cannot time sub-millisecond work (*Shipilev’s “Nanotrusting the Nanotime”, the LMAX blog, many Java write-ups*). None of these connects it to results being silently thrown away inside a benchmark’s percentiles.
- **Negative latency in practice.** Negative latencies and negative span durations are reported often: a Kafka mailing-list thread, Jaeger and OpenTelemetry users, and a 2026 Kafka monitoring guide. They are almost always blamed on clock skew; RabbitMQ PerfTest’s own documentation warns about clock sync only. Tools deal with them by reporting “not a number” (*Kafka’s consumer latency metric*), clamping to zero (*Splunk’s OpenTelemetry collector*), dropping them, or breaking dashboards.
- **Scheduler delays.** Kernel developers and practitioners report latency spikes and regressions tied to the EEVDF scheduler’s slices (*LWN 2026; a 2026 write-up on fixing EEVDF regressions at Meta*). None connects those delays to wrong timestamps in latency tools, and none gives a law.
- **Bottom line.** Practitioners know the symptoms. None of the sources we found ties them to scheduling or to a predictive law, and none compares broker latency tools against each other. T5 makes this a count rather than an impression. [D3-8]

10 Predictions, frozen before running

ID	Prediction	Counts as confirmed if
P1	The cliff follows the slice. For each slice s , the trip at which the negative rate falls halfway from the plateau (<i>the level at $0.9s$</i>) to the floor (<i>the level at $2(s + h)$</i>) lies between s and $s + h$. Across the six slices, after each campaign’s offset is removed through the anchor slice, [D4-3] that halfway point rises one-for-one with s .	In at least 5 of the 6 slices the halfway point is inside the band. The straight-line slope against s lies between 0.8 and 1.2, with a 95% interval that excludes 0.5 and 1.5.
P2	The cliff’s width follows the tick. The drop from 90% to 10% of the plateau is about four times wider at HZ=250 than at HZ=1000.	The width ratio’s 95% interval overlaps 2.5–5.5 and excludes 1.
P2b	And about ten times wider at HZ=100. [D3-7]	The width ratio’s 95% interval overlaps 6–14 and excludes 1.
P2c	The width is proportional to the tick. [D3-7] Across the three kernels, the fitted cliff width against tick length is a straight line through zero with slope one.	The slope’s 95% interval overlaps 0.8–1.2 and excludes 0.5; the intercept’s 95% interval includes 0.
P2d	The tick does not move the cliff’s start. [D3-7] The fitted start stays at the slice whatever the tick.	In all three kernels, at both slices, the fitted start is within 0.25 ms of the slice.
P3	Load raises the plateau, not the cliff. At the 3 ms slice, [D4-4] between 50% and 88% load the plateau height rises, but the halfway point moves by less than 0.25 ms.	Plateau increase with a 95% interval above zero; shift of the halfway point under 0.25 ms.
P4	Go-first priority removes the plateau. At the 3 ms slice, [D4-5] it cuts the plateau rate at least 5 times wherever the ordinary plateau is at least 2%.	Point estimate ≥ 5 , 95% interval above 2.

ID	Prediction	Counts as confirmed if
P5	Measurement checks. [D3-5] (a) Arrival minus sending is never below zero. (b) The receiver-only delay lengthens the trip in a way we can rely on. (c) It leaves the “got it” delay unchanged.	(a) 0 negatives per run. (b) In every session’s calibration, the slope’s 95% interval lies above 0.5, and at every step the calibration’s 95% interval lies within 0.3 ms of it (<i>from the scatter of the runs about the line, or within the steps where the step medians are joined</i>); each run’s and each step median’s distance from it is reported. [D6-1] (c) Equivalent within ± 0.10 ms on the median and $\pm 10\%$ on the p99, at every delay. Otherwise the session is analysed with each run’s own “got it” distribution, and the departure is reported. A median shift larger than a quarter of the added delay stops the session.
P6	A recording predicts the rate , with nothing fitted, within a factor of 1.5, and not systematically too low.	At least two thirds of setups within 0.67–1.5, and the median ratio at least 0.8.
P7	Default slices follow the core-count rule , computed from the per-CPU constant read off each machine (<i>on the first Azure machine: 1.4, 2.1 and 2.8 ms at 2, 4 and 8 cores</i>), and the cliff follows them. [D3-3]	Read-back values match; P1’s band holds at each.
P8	Language (Kafka). [D4-9] Java shows the same cliff and priority effect as Python: the rate at $s+0.5h$ lies between its levels at $0.9s$ and $s+1.5h$, and go-first cuts the plateau at least 5 times. Python’s rate is at most 1.5 times Java’s at matched setups.	All hold at the 95% level, on each machine pair.
P9	New hardware. [D4-14] On the Arm pair the cliff follows the slice: at each of the three slices the halfway point lies between s and $s+h$, and it rises with the slice, after the anchor correction.	The halfway point is inside the band at all three slices, and its straight-line slope against s lies between 0.8 and 1.2.
<i>Why the trip does not grow one-for-one (M0). [D3-6] Each explanation is kept or dropped by its test, and more than one may hold. “The departure” is how far the measured slope is from one, times the added delay.</i>		
M-H1	The network. The delay that reaches the receiver’s network card differs from the delay set.	Kept if the arrival shift in the packet capture differs from the set delay by more than 0.05 ms.
M-H2	The receive loop. A longer delay changes whether a new message finds a request already waiting at the broker, or has to wait for the next one.	Kept if a replay of each run’s message times against its logged request cycle reproduces the measured slope within 0.05.
M-H3	The wake-up. With a delay, the receiving thread has gone to sleep when the reply arrives, and must be woken and scheduled.	Kept if the change in the thread’s median wake-up delay between 0 and 8 ms accounts for at least half of the departure.
M-H4	The TCP acknowledgement (<i>expected for Redis</i>). Delaying the broker’s replies also delays its acknowledgements, which can hold the receiver’s next request back.	Kept if requests held for an acknowledgement appear in the capture and account for at least half of the departure.
T-P1	Low, chops after measuring (Kafka EndToEndLatency). Percentiles equal the true value rounded down to whole ms: 0 for every step below 1 ms. The average is within 2% of our reference.	All staircase steps match.

ID	Prediction	Counts as confirmed if
T-P2	Low, reads a ms clock twice, keeps all (ProducerPerformance, emqtt-bench). With send times spread evenly, the average is within 0.05 ms of the truth, and the median flips from 0 to 1 at a true 0.5 ms. With sends paced in step with the tick, the average can be off by up to 0.5 ms.	Both patterns seen at the predicted steps.
T-P3	Low, drops ≤ 0 (OpenMessaging end-to-end). For trips under 1 ms, the median prints 1.0 ms at every step, and retention is about the trip divided by 1 ms.	Median constant below 1 ms; retention within 5 points of prediction.
T-P4	Low, drops negatives, keeps zeros (Pulsar perf). The median is 0 below a true 0.5 ms. Under a forced offset, the average rises by exactly what dropping the negatives removes.	Average shift within 10% of the computed value.
T-P5	Medium (wrk2). It tracks 0.1 ms steps within 0.05 ms at the median. Forced negatives are not recorded, or the run aborts.	Both behaviours as predicted.
T-P6	High, keeps all (RabbitMQ PerfTest, NATS CLI). They track 0.1 ms steps within 0.02 ms. A forced offset shifts every value by the offset, and negatives show in the minimum and pull the average down.	Shift within 0.02 ms; negatives present.
T-P7	Every “read first” tool behaves as its class predicts, in T1 and T2.	At least 90% of the predicted behaviours observed.
T-P8	The documentation stays silent. [D3-8] Where the source rounds, drops, clamps or replaces values (<i>T-P1 to T-P5</i> , <i>fio</i> , <i>btt</i>), the documentation does not say so.	Documented in at most a third of those tools.

11 Statistics: how many runs, and in what order

How many rounds, from each campaign’s own test. [D4-2] The number of rounds is not fixed in advance; the rule that sets it is. Before a campaign’s main runs, and after its machine pair’s spread pilot (*P0*), the campaign is simulated 1,000 times on made-up data:

- **The made-up data follow the law exactly.** They use the plateau height and floor that the pair’s *P0* and *B0* measured at the campaign’s load; the campaign’s own slices, trips, loads and tick; the planned number of messages per run; and run-to-run variation, on the logarithm of the rate, equal to the spread *P0* measured on that pair for that backend. Shifts between campaigns and between sessions are added, with a standard deviation of 0.1 ms in trip and 20% in plateau height.
- **Each simulated campaign goes through the real analysis** and the same “counts as confirmed if” rule as the real data (Section 10).
- **Rounds** are the smallest number, at least 4, for which the rule confirms the prediction in at least 80% of the simulations, and in at most 5% of simulations where the prediction is false in the way its falsifier states: a cliff that stays put as the slice changes (*P1*, *P9*); a width that does not change with the tick (*P2 to P2c*); a halfway point that moves 0.5 ms between 50 and 88% load (*P3*); no go-first effect (*P4*); a cliff that stays put as CPUs are switched off (*P7*); Python’s rate twice Java’s (*P8*).
- **Blocks of several campaigns** (*A1*, *A2*, *A4*) are simulated as a whole, and every campaign of a block runs the same number of rounds.
- **Ceiling:** 40 rounds. If 40 rounds cannot reach that power, the campaign runs at 40 and its prediction is reported as underpowered, as written in the campaign’s log before it runs.
- **Fixed numbers where no prediction is tested:** *C0* runs 2 rounds per session and 4 in *S0-1*; *B0* runs 3; *P0* runs 5; *M0* runs 10. Tool campaigns run 4 rounds: their predictions are patterns

(rounding, dropping, a median stuck at 1 ms) that one round already shows, and the repeats measure how stable each pattern is.

- **Why not a fixed floor.** Version 3 required at least 15 runs per setup, from old setups whose run-to-run spread was about as large as the rate. The pilot’s repeat runs varied far less (*a standard deviation of 0.14 to 0.23 on the log rate*), and a simulation of the actual decision rule measures power directly. The simulation code is tested on made-up data before its first use, and its seed is recorded.

Spread pilot (P0): 5 rounds of 16 setups at 75% load on every machine pair, before that pair’s main campaigns. [D4-10] They are used only to estimate run-to-run spread, never to test a prediction. The instrument pilot in Section 5 is a different thing.

Machine pairs. [D4-10]

1. **One pair per test.** Every prediction is tested on runs from one pair. A1, A3 and A7 run on the first x86 pair; A5 and all fifteen A2 campaigns on one x86 pair; A4 on the Arm pair; each tool on one pair. A8 runs once on an x86 pair and once on the Arm pair, and P8 is judged on each.
2. **Every pair opens with its own B0 and P0** and reads back its settings, so the pairs can be described against each other and the repeats rule uses that pair’s own spread.
3. **Every run records its pair.** No estimate pools runs from different pairs.
4. **At the same time:** campaigns on different pairs may run at once; campaigns on one pair never overlap.

Random order, so a passing server mood cannot fake an effect:

1. **Rounds.** List every setup of the campaign. One round contains every setup once. Repeat for as many rounds as the rule gives.
2. **Shuffle.** Within each round, shuffle the order with a random generator whose seed is written in the run log.
3. **No back-to-back repeats.** If the last setup of one round equals the first of the next, reshuffle.
4. **Spread over time.** [D4-1] The campaigns of one block run in an order drawn with a recorded seed, over at least two days and at different times of day (*morning, afternoon, night*).
5. **Settings that need a reboot, a new machine or another pair.** Tick, kernel, core count, machine type and machine pair cannot be shuffled run by run. They are *sessions*: the order of sessions is randomized, and each session type is repeated on at least 2 different days. Kernel sessions are counterbalanced across days. [D3-7] The slice *can* be changed without a reboot, so it is shuffled run by run inside a session like everything else. The analysis treats session as its own source of variation (*split-plot design*).
6. **Failed runs.** [D5-2] If a run fails mechanically, or a condition it was meant to have did not take (*Section 13*), it is logged with the reasons and re-queued at a random later position in the same session, 3 attempts in all. A run whose conditions held but that gives a surprising number is a result, never a re-run.

Integrity checks, the same for every arm. [D5-2] Before a run counts, it must pass all of these. Section 13 says when each is checked and what follows a failure. Runs that fail are counted, reported and never silently dropped.

- Arrival minus sending is never negative.

- The session’s delay calibration passed P5(b), and the run shows no gross “got it” departure ($P5(c)$). [D3-5]
- At least 99% of the planned messages are sent after the warm-up, and at least 99% of those arrive. [D5-2]
- The achieved send rate is within 2% of target.
- Measured load is within 3 points of target, from at least 10 samples taken while messages are sent.
- The slice and tick read back as intended, before and after the run.
- The added delay, measured by ping beyond the same ping taken with no delay just before the run, is within 0.25 ms plus 5% of the delay set. [D5-2] [D6-5]
- Before the run, the machine is as its session set it up: the receiver’s address only in its namespace, the broker answering, and no package manager running. [D6-8]
- The clients ran with the campaign’s settings, as their own logs state: Kafka’s producer with up to 64 requests in flight, Redis’s consumer acknowledging in batches of 200. [D6-6]
- The clock’s offset is logged before and after the run.
- The first 30 seconds are discarded.

How the cliff is measured, fixed before the data. For each slice, the curve of negative rate against trip is summarized two ways:

- **Fitted shape:** fit the plateau, straight fall and floor to the runs, and read off where the fall starts and how wide it is.
- **Free shape:** fit a smooth curve that is only required to go down, and read off the halfway point and the 90%-to-10% width.

The law passes only if both agree with the prediction. Intervals come from resampling whole rounds (*bootstrap*). **Each run sits at the trip it actually had**, its own measured median trip, not the trip it was aimed at. [D3-5] As a secondary analysis, fixed now, every message is also placed at its own trip within its session, which traces the cliff inside runs as well as across them.

Removing campaign offsets through the anchor. [D4-3] In A1 and A4, a campaign’s offset is its anchor slice’s halfway point minus the mean of the anchor’s halfway points over the block’s campaigns; for plateau height, the same on the log scale. Each campaign’s other halfway points and plateau heights are corrected by its offset before P1 or P9 is judged. Both predictions are also reported without the correction.

Analysis, fixed before the data.

- **Unit of analysis:** the run.
- **Comparisons:** ratios on the log scale, allowing for session and round.
- **Pairs and campaigns:** [D4-10] machine pair, campaign and session are sources of variation; nothing pools runs across pairs.
- **Several predictions in a family:** tested together with Holm’s correction.
- **Calibration** ($C0$): [D3-5] for each backend and session, a straight line of run median trip against measured added delay, with a test of the line against the per-step medians (*lack of fit*). Intervals come from resampling whole rounds. If the line fails the test, the per-step medians are joined by straight segments instead, and the shape is reported.

- **“Got it” equivalence** ($P5(c)$): [D3-5] two one-sided tests at 90% at each delay step against zero delay, with margins of ± 0.10 ms on the median and $\pm 10\%$ on the p99.
- **Code first:** the analysis code is written and tested on made-up data before the first real run it analyses.
- **Frozen predictions:** this plan is frozen as Section 15 describes, before the first campaign.
- **Every run tracked:** failures included, as in all our earlier campaigns, each with its verdict, its reasons and its fingerprints. [D5-4]

12 The campaigns [D4-1]

Stage	Campaigns	Question	Runs	Hours each	Hours
0 Instrument	4: S0-1 delay staircase; S0-2 M0; S0-3 B0 and P0; S0-4 B0 and P0 on a second x86 pair, if one is used	how the delay moves the trip, why, and how much runs vary	414	3.5–7.4	25.2
1 Slice (<i>A1</i>)	6: per backend, slices of 0.75, 3 and 4.5 ms; 1.5, 3 and 6 ms; 2.25 and 3 ms	does the cliff follow the slice?	592	4.7–6.7	36.0
2 Load (<i>A3</i>)	2: one per backend	does load leave the cliff where it is?	216	6.5	13.1
3 Go-first (<i>A7</i>)	1	does priority remove the plateau at the cliff?	72	4.5	4.5
4 Core count (<i>A5</i>)	2: one per backend	does the kernel’s default slice set the cliff?	264	8.4	16.9
5 Tick (<i>A2</i>)	15: the build, 12 sessions, 2 bridge sessions	does the cliff’s width follow the tick?	682	3.5–5.2	55.5
6 Arm (<i>A4</i>)	5: the Arm instrument campaign, then 2 per backend	does it hold on another processor?	430	4.7–7.4	26.3
7 Client (<i>A8</i>)	2: an x86 pair and the Arm pair	is it the Python client?	240	7.2	14.5
8 Tools (<i>T1–T4</i>)	10: one per runnable tool	what do the tools report?	680	4.7	47.2
T5 audit	desk work	does anyone say so?	–	–	–
Total	47		3,590		239.2

Runs and hours are at the floor of 4 rounds. A run takes 3.5 minutes, as the pilot measured (*warm-up, at least 5,000 messages, settling and checks*); each campaign adds 0.25 hours of setup, and each reboot 0.5 hours. At 6 rounds the total is 4,826 runs and 312 hours. M0’s second Redis arm adds 30 runs (*D6-7*); a calibration’s second stage, where one is needed, adds 24 runs to its session (*D6-2*).

Which pair runs what. With one x86 pair, every x86 campaign runs on it and S0-4 is not needed. With two x86 pairs, the first runs S0-1 to S0-3, A1, A3, A7, A8’s x86 campaign and five tool campaigns; the second runs S0-4, A5, all of A2, and the other five tool campaigns. The Arm pair runs A4 and A8’s Arm campaign.

How long. At about 20 hours a day: one x86 pair and then the Arm pair, about 11.5 days; two x86 pairs at once and then the Arm pair, about 7 days; all three pairs at once, about 5 days. [D5-7] Each new pair’s shakedown adds a few hours to these. The rules of Sections 11 and 13 hold in every case. [D5-1]

Money. About EUR 97 of machine time at list prices with all three pairs in Sweden Central (*EUR 126 at 6 rounds*), plus about EUR 26 a month per pair for disks and fixed addresses while the pair exists. [D5-7]

Order of work:

1. **Now, with the machines stopped.** Freeze this plan, together with the code that judges each run, stops a campaign, watches the pairs and copies the data, which is written and tested before the freeze. [D5-2] Write, and test on made-up data, the code for M0’s recording, the tick checks, the simulated-power rule and the law analysis. Read the “read first” tools and freeze their classes. Do the T5 audit. Prepare the kernel builds and the Java client.

2. **Stage 0.** On the first x86 pair: S0-1, then S0-3, then S0-2 once M0's recording is tested. On a second x86 pair, if one is used: its shakedown, then S0-4 and A2's build. The Arm pair's shakedown comes before its instrument campaign. [D5-1] Stage 0 starts again from the beginning under version 6. After any change to the code, the first x86 pair runs alone until its shakedown and first calibration pass, and then the others start. [D6-12]
3. **Rounds.** Each campaign's rounds come from Section 11 and are written in its log before it runs.
4. **The campaigns,** in the order of the table above on each pair, with each block's campaigns in their seeded order. Each campaign's runs are copied and checked before the next campaign starts on its pair. [D5-4]
5. **Analysis and report.** Every prediction reported as confirmed or not, with every run listed.

If money or time runs short: the slice and tick campaigns first, since they are the law; then A4 and the tools; then the rest.

13 How the campaigns are run, checked and stopped [D5-1]

Campaigns run unattended, often at night, on up to three machine pairs at once. So the rules for running them are fixed here, before any run, and a program applies them, not a person looking at results.

Lanes. [D5-6] Each machine pair is a *lane* (*a driver and its broker, with their own queues and logs*). A campaign runs on one lane from its first run to its last, and the campaigns of one lane never overlap. Two lanes share no machine, broker, network link or disk, so a run on one cannot disturb a run on the other. Every lane runs the same version of the code, and every run records its lane, its driver and that version (*the commit*) next to its files.

A new pair's shakedown. [D5-1] Before a pair's first campaign, the pilot of Section 5 runs on it. Its scheduler-settings, receiver-only-delay, load and never-negative checks must pass, and a pair that fails one runs nothing until the cause is found. The receiver-only delay is checked by the broker's own capture of the pilot's pings, within 0.05 ms, and by ping end to end, within 0.25 ms plus 5%; [D6-3] with no delay, the two paths must agree within 0.25 ms, and both machines' network and timing facts are kept. A pair that passed its shakedown repeats this network part in every later session, because a start after a stop can put a machine on another physical host. [D6-4] The pilot's harness-delay check is replaced by the pair's own C0, as on the first pair (*D3-5*). Its go-first result is recorded but is not a gate, because A8 tests go-first as a prediction.

Every run, as it ends. [D5-2] A program on the driver checks each run the moment its messages stop, writes every check next to the run with the value found and the limit it was held to, and gives one of three verdicts:

Verdict	When	What follows
Counts	Every integrity check of Section 11 passes.	The run enters the analysis.

Verdict	When	What follows
Repeated	A condition the run was meant to have did not take: fewer than 99% of the planned messages sent after the warm-up, or fewer than 99% of those arrived; the send rate more than 2% from target; the load more than 3 points from target, or fewer than 10 load samples while messages were sent; the slice or tick different after the run from what was set; the added delay, measured by ping beyond a zero-delay ping taken just before, more than 0.25 ms plus 5% from the delay set; [D6-5] the machine not as its session set it up before the run; [D6-8] a client not set as every campaign sets it; [D6-6] the clock's offset not logged before and after; or a step that failed, or files that cannot be read.	The run keeps its files and its line in the ledger, marked failed with its reasons, and a copy is queued at a random later place in the campaign, 3 attempts in all. A run whose third attempt fails is abandoned and reported.
Stops the campaign	The instrument is in doubt: a message arrived before it was sent ($P5(a)$), or the run's "got it" median moved from the session's zero-delay median by more than a quarter of the delay added ($P5(c)$).	The run is recorded as failed, and nothing more runs on that pair until the cause is found and recorded with the campaign. If the cause is in the code every pair runs, the other pairs stop too.

Nothing a run measured makes it a repeat: a run whose conditions held is a result, whatever its number. The "got it" comparison needs the session's calibration, so it applies to every run after the session's C0; within C0 itself, P5(c) is judged when the calibration is fitted.

When attempts keep failing. [D5-3] After every attempt the ledger is read again. The campaign stops itself when the last 3 attempts all failed, or when more than 20% of the last 20 failed, once at least 10 have finished. A machine that has gone wrong then costs minutes, not a night.

Void sessions. [D6-10] A session broken by a fault of the machines or of our code, or stopped to fix one, before its calibration's gate is void. Nothing in it is deleted: its files and ledger are copied and fingerprinted like any campaign's, a note beside them gives the cause, and the report lists it. It is never analysed. A session whose calibration fails its gate ends too, and the pair starts a new one with a new C0; no session is held open while a rule changes. [D6-11]

Watching every pair. [D5-5] A watch on a computer outside the testbed reads every pair every 5 minutes. It changes nothing on the machines, except that it deallocates a pair whose machines have been idle for 20 minutes, so that the pair bills only for its disks. It raises:

- **alerts**, which need a person now: a campaign that stopped itself, or a run judged to stop one; a message that arrived before it was sent; a campaign with no progress for 20 minutes; new failed runs; a broker that is not running; a disk 80% full, or less than a tenth of memory free; a machine that does not answer; one queue running on two pairs;
- **warnings**, worth a look: a repeated run; CPU time taken by other tenants above 5% (*steal*); a clock more than 1 ms off; load far below its setting; pairs running different versions of the code;
- **notices**: a finished campaign, whose runs are to be copied before the next campaign starts on that pair.

Every look is kept in a dated log.

Keeping the data. [D5-4] A campaign's runs exist only on its driver until they are copied. When a campaign finishes, and before the next starts on that pair, its ledger and every file of every attempt, done or failed, are fingerprinted on the driver (*SHA-256: a 64-character code that changes if any byte of the file changes*), packed, copied, and checked file by file against their fingerprints. The fingerprints are

kept with the data, with a record of the machine, the code version and the time of the copy. A copy that fails its check is made again from the driver, where nothing is deleted. A pair is deleted only after all its campaigns are copied and checked. What a driver measured outside any campaign (*a pilot, a void session, a probe of the instrument, a log*) is copied and checked the same way, and before a pair is deleted its whole folder of runs is copied once more. Every run also keeps the TCP counters of the driver, the receiver's namespace and the broker, and the broker's own log for the run. [D6-10] Every run writes into a folder of its own, named after its campaign and its place in it, and a campaign that finds that folder already there stops. [D6-9]

14 What could go wrong, and what it would mean

- **P1 fails** (the cliff does not follow the slice). The “law” is only a description of our old machines. That is still worth reporting, and it would redirect the paper.
- **P2 to P2c fail** (the width does not follow the tick). The wait is not “slice plus up to one tick”. The plateau-cliff shape may survive with a different width rule. If only P2c fails, the width grows with the tick but not in proportion, and something else sets it at long ticks.
- **P2d fails** (the tick moves the cliff's start). The slice is enforced only on tick boundaries, and the law's start becomes the slice rounded up to a tick. [D3-7]
- **P3 fails** (load moves the cliff). Waiting is more than one protected slice under heavy load, for example several tasks queued ahead. That would point to the classic time-sharing queueing laws, which do predict this.
- **P8 fails on Java**. The effect is tied to Python's lock or our client. That narrows Mode 1 to Python-style clients.
- **The slice setting does not stick**, because the kernel resets it when cores change. Read it back before every run and discard runs where it drifted.
- **The delay's effect is not a straight line (C0)**. [D3-5] The per-step medians are joined instead, and the shape is reported. If a longer delay can give a shorter trip, the receiver-only delay cannot place trips, and that machine's main runs do not start.
- **No explanation survives M0**. [D3-6] The slope is reported as an open property of the clients. The law's tests still stand, because each run is analysed at the trip it actually had.
- **HRTICK is on in a kernel**. [D3-7] It is switched off, or the kernel rebuilt, before any tick run. A tick test with it on would be void.
- **A new kernel does not boot**. [D3-7] The machine falls back to the stock kernel, the failure is logged, and no run is lost.
- **The two backends disagree**. Both are reported, and the law's claim is narrowed to where it holds.
- **The machine pairs differ** [D4-10] in baseline trip or run-to-run spread. Each pair's tests stand on their own, with that pair's own repeats, and the difference is reported.
- **A simulation asks for more than 40 rounds**. [D4-2] The campaign runs at 40, and its prediction is reported as underpowered, as its log says before it runs.
- **The anchor moves between campaigns** [D4-3] by more than 0.25 ms at its halfway point. The movement is reported, and P1 is given both with and without the anchor correction.
- **A campaign stops itself**. [D5-3] Nothing more runs on that pair until the cause is found. Runs finished before the stop stand if the cause did not touch them; if it did, they are repeated, and

both the cause and the repeats are reported.

- **A pair is lost in the middle of a block** [D5-4] (*a machine fails, or its disk is lost*). The campaigns already copied are safe. Because no test pools runs from two pairs, the interrupted block runs again in full on a rebuilt pair, after its shakedown, B0 and P0, and the interrupted runs are reported.
- **A pair's runs vary too much for its calibration.** [D6-2] Two more rounds run; if the calibration is still too loosely known, the session ends, is reported with its spread, and the pair starts a new one.
- **The two paths differ with no delay** [D6-4] by more than 0.25 ms. The pair runs nothing that session; it is stopped and started again, which can move it to another physical host, and each attempt is reported.
- **A client does not take its setting.** [D6-6] Its runs are repeated, and after three failed attempts in a row the campaign stops itself (*D5-3*); the cause is found before it starts again.
- **Kafka runs keep pausing** [D6-10] for fractions of a second. The pauses are reported with the TCP counters and the broker's log of each run; the analysis uses medians and per-message placement, which a few paused messages barely move.
- **A copy does not match its fingerprints.** [D5-4] It is made again from the driver. A file that still does not match is reported, and its run is listed as lost, never silently dropped.

15 How this plan is frozen [D4-13]

A pre-registration is only worth something if a reader can check that it was written before the data. This plan is frozen by committing it in full, its source and this PDF with their SHA-256 fingerprints, to the **freezes** folder of the project's public repository, before the first Stage 0 run. The code that applies Section 13 is in the same repository, and every run records the version of it that ran. [D5-6] A later change is a new, numbered freeze whose first section lists what changed and why, never an edit of this one. This version is the second freeze (*freeze 02*). [D6-12] Results are reported against the freeze in force when they ran, including every prediction that failed.